

**ПРАВИТЕЛЬСТВО РОССИЙСКОЙ ФЕДЕРАЦИИ
ФГАОУ ВО НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
«ВЫСШАЯ ШКОЛА ЭКОНОМИКИ»**

Факультет компьютерных наук
Образовательная программа «Прикладная математика и информатика»

Отчет о программном проекте на тему:

KV-store с гибридными протоколами консенсуса

Выполнил:

Студент группы БПМИ235

А.Ю.Коновалов

И.О.Фамилия

01.04.2026

Дата

Принял руководитель проекта:

Трушин Дмитрий Витальевич

Имя, Отчество, Фамилия

доцент, к.ф.-м.н.

Должность, ученое звание

ФКН НИУ ВШЭ

Место работы (Компания или подразделение НИУ ВШЭ)

Москва 2026

Содержание

1	Введение	3
2	Используемые технологии	4
3	Описание применения генеративной модели	4
4	Обзор альтернативных подходов	4
5	Обзор литературы	5
6	Общая структура	5
7	Message-Driven System	5
7.1	Варианты MDS	7
7.2	MDS в данной работе	7
7.3	Формат события, контракт полей	7
7.4	Время и порядок обработки	7
7.5	Контекст выполнения	8
7.6	Регистрация узлов и адресация	8
7.7	Зачем такое ядро	8
8	Симулятор	9
8.1	Новые компоненты	9
8.2	Модернизация контекста <code>SimulationContext</code>	9
8.3	Доставка через узел <code>Network</code>	9
8.4	Две перегрузки <code>SendLocal</code>	10
8.5	Сборка сценариев	10
8.6	Вывод	11
9	Распределенное KV-хранилище	11
9.1	Локальный контракт	11
9.2	Инвариант согласованности	11
10	Основы консенсуса Raft	11
10.1	Контекст в симуляторе	11
10.2	Типы состояния узлов в Raft	12
10.3	Что решает протокол и какие у него гарантии	12
10.4	Сообщения протокола и команды KV	12
10.5	Функциональная роль узлов	12
10.6	Выборы и условие голоса	13
10.7	Репликация и коммит	13
10.8	Данные в узле и таймеры	13
11	Гибридный протокол на базе Raft	13
12	Сборщик метрик	13
12.1	Интерпретация в симуляторе	14
12.2	План клиентских операций и задержки	14
12.3	Сборка сценария	14
12.4	Связь с хаос-тестами	14
13	Эксперименты	14
13.1	Множественный запуск и сборка статистики	14
14	Заключение	16

Аннотация

В данной работе представлен дискретно-событийный симулятор для моделирования и тестирования алгоритмов в распределенных системах. Разработанная программная среда позволяет создавать виртуальные узлы сети, моделировать передачу сообщений с учетом задержек и потерь, а также эмулировать отказы узлов и разделения сети. На этой основе реализованы Raft и гибридный протокол консенсуса для распределенного key-value хранилища, проведено сравнение по метрикам. Исходный код доступен в репозитории [7].

Ключевые слова: C++, распределенные системы, дискретно-событийный симулятор, алгоритмы консенсуса, симуляция сетевых протоколов.

1 Введение

При проектировании и модификации вычислительных систем, обрабатывающих пользовательские запросы и возвращающих им ответы, долгое время основным путем роста производительности оставалось *вертикальное масштабирование* — наращивание мощности одного узла за счет замены его компонентов (процессор, память, дисковая и сетевая подсистема) на более производительные. Такой подход прост в эксплуатации, но связан с принципиальным ограничением: отказ критически важного компонента на единственной машине приводит к недоступности всего сервиса целиком.

Наряду с вертикальным масштабированием все шире стало применяться *горизонтальное масштабирование*, т.е. увеличение числа независимых вычислительных узлов и распределение нагрузки между ними. Это позволяет наращивать пропускную способность и при коммуникации узлов по единому протоколу повышать *отказоустойчивость*: свойство, при котором выход из строя отдельной машины не обязан означать полную остановку обслуживания. В современных распределенных системах потребность в таком сочетании подходов — вертикального на уровне узла и горизонтального на уровне *кластера* (группы вычислительных машин, способных обмениваться сообщениями) — стала практически обязательной.

Возникает и иная проблема: копии состояния на разных машинах могут расходиться, и при обращении к разным узлам клиенты могут получать противоречивые ответы. *Линеаризуемость* требует, чтобы система вела себя так, как если бы существовала одна последовательная копия данных, согласованная с порядком завершения операций в реальном времени.

Для обеспечения отказоустойчивости при репликации состояния и для соблюдения линеаризуемости необходимо, чтобы машины в кластере коммуницировали между собой согласно определенному протоколу. К таким протоколам относятся *протоколы консенсуса*, обеспечивающие согласованное принятие решений несмотря на задержки и отказы.

Существует несколько алгоритмов консенсуса, самые яркие представители это Paxos и Raft. Их краткий обзор есть в главе с литературой, а также ниже будет подробно рассмотрен алгоритм Raft, являющийся основой для гибридного алгоритма.

Удобной моделью системы с реплицируемым состоянием является *конечный автомат состояний* (*state machine*): у системы есть текущее *состояние*, а входящие *команды* задают детерминированные переходы — по одной и той же последовательности команд все корректные копии приходят к одному и тому же состоянию. В распределенном варианте узлы согласуют *общий порядок применения команд* (часто через упорядоченный журнал), после чего локально исполняют их в этом порядке — такой подход называют *репликацией конечного автомата* (*replicated state machine*).

KV-store (хранилище "ключ - значение") — частный случай такого автомата: состояние можно представить как отображение ключей в значения, а команды записи и удаления (**SET**, **DEL** и т.п.) — как переходы, изменяющие это отображение. Операции чтения (**GET**) состояние не меняют. В схеме *журнальной репликации* все изменения данных проходят через *согласованный журнал* — упорядоченную последовательность команд записи, одинаковую на всех корректных репликах: порядок записей в журнале выбирается и фиксируется протоколом консенсуса, после чего команды последовательно применяются к локальному состоянию KV-store. Поэтому **GET** обычно не добавляется в этот журнал как новая согласуемая запись. При этом ответ на чтение должен отражать уже принятые и примененные обновления и оставаться согласованным с состоянием автомата на выбранной реплике (в той же модели линеаризуемость действительно работает благодаря согласованности порядка записей в журнале). В рамках этой работы алгоритмы консенсуса рассматриваются именно как алгоритм, гарантирующий согласованную репликацию состояния KV-store в кластере.

Цель работы — разработка детерминированного симулятора для тестирования алгоритмов консенсуса в распределенных системах. Также реализация на его основе гибридного протокола консенсуса для KV-store, а также сравнение этих двух алгоритмов.

Задачи работы:

1. Реализовать *Message-Driven System* (MDS) с кастомной логикой упорядочивания событий.
2. Написать unit-тесты на реализованную MDS.
3. Поверх MDS реализовать дискретно-событийный симулятор с задержками и потерями по сети, с частичными партициями (разделениями сети на несвязные компоненты), а также с падениями узлов.
4. Покрыть тестами симулятор, а также предоставить пример использования симулятора.
5. Имплементировать KV хранилище, объектом которого будут оперировать алгоритмы консенсуса.
6. Имплементировать алгоритм консенсуса Raft.
7. Реализовать юнит тестирование и стресс-тестирование алгоритмов консенсуса. Проверить корректность реализованного алгоритма Raft.
8. Реализовать Metric collector, способный агрегировать внутри себя метрики, по которым будут сравниваться алгоритмы Raft и гибрид.
9. Имплементировать гибридный алгоритм на основе Raft, описать главные улучшения по сравнению с vanilla Raft.
10. Проверить корректность гибрида на юнит и стресс тестах.
11. Сделать ансамбль запусков Raft и Hybrid Raft, собрать метрики, сравнить и сделать вывод.

В рамках этой курсовой работы удалось достичь всех поставленных целей. По собранным метрикам гибридная реализация Raft в описанных сценариях показывает лучшие показатели по сравнению с базовой.

Исходный код проекта, тесты и конфигурация сборки доступны в открытом репозитории [7].

2 Используемые технологии

Реализация ведётся на C++20. Сборка описана в CMake: отдельные цели для библиотек ядра, симулятора, Raft и гибрида, исполняемых тестов и примеров. Зависимости (в частности Catch2) подтягиваются благодаря Conan. Тесты написаны с Catch2.

3 Описание применения генеративной модели

В рамках данной работы использовался сервис ChatGPT [4] только для оформления таблицы сравнения метрик; результат на рисунке 3.

4 Обзор альтернативных подходов

Для тестирования корректности реализации распределенных алгоритмов применяется симуляция кластера, с моделированием частичного отказа узлов и сети. Симуляции в свою очередь делятся на два класса:

1. *Emulation based симуляторы*, где каждый узел запускается и взаимодействует со средой в реальном времени. Такой симулятор нужен для нагрузочного тестирования.
2. *Event based симуляторы* (дискретно-событийные), где время идет дискретно, и симулятор итерируется по событиям в распределенной системе.

В рамках данной курсовой работы *дискретно-событийный симулятор* реализован и используется для тестирования протоколов.

Также важная часть любого симулятора - воспроизводимость поведения и детерминированность. Они позволяют воспроизводить сценарии выполнения: при фиксированном seed генератора случайных чисел последовательность событий идентична между запусками. Это критически важно для отладки — найденный баг можно воспроизвести и исправить.

5 Обзор литературы

Raft [3] — алгоритм консенсуса, спроектированный для понятности. Разделяет задачу на выбор лидера, репликацию лога и безопасность. Будет использован как основа для гибридного протокола

PBFT [1] — алгоритм консенсуса, устойчивый к *византийским отказам* - состоянию системы, когда нода может не только падать, но и вести себя произвольно, в том числе злонамеренно. Работает при $f < \frac{n}{3}$ злонамеренных узлов, где n - общее количество узлов, а f - количество нестабильных или упавших узлов.

Paxos [2] — классический алгоритм консенсуса Лэмпорта. Лежит в основе многих практических систем. Его считают трудным для практической реализации, а исходные изложения автора Leslie Lamport местами допускают разную интерпретацию. В результате реализации Paxos в системах часто отличаются друг от друга и не всегда в полной мере формализуют краевые случаи.

Distributed Systems [6] — учебник Таненбаума, дает знания и контекст по распределенным системам.

Enterprise Integration Patterns [?] — книга Hohpe и Woolf, в которой они систематизировали подход к построению message-driven systems: компоненты обмениваются сообщениями через каналы и очереди, а не вызывают друг друга напрямую. Там изложены типовые паттерны такой архитектуры.

Лекции по C++ (MDS) [5] — учебный PDF по C++. В нем последовательно излагается message-driven взгляд на построение программ.

6 Общая структура

Программная среда для экспериментов с консенсусом спроектирована *слоями*: каждый верхний уровень опирается на контракт нижнего и не дублирует механизм упорядочивания событий.

На схеме выделены четыре уровня (снизу вверх - от ядра к наблюдаемости за прогонами):

1. **Message-Driven System (MDS)** Ядро с единой приоритетной очередью событий и диспетчером: все асинхронные действия сводятся к последовательной обработке событий с детерминированным порядком при совпадающем времени. Этот слой не знает ни о сети, ни о Raft, ни о специальных нодах - только об узлах-обработчиках и метках времени.
2. **Дискретно-событийный симулятор** Настройка над MDS: модель сети, включая задержки, потери, партиции, пул узлов, таймеры, классифицированные события доставки и, при необходимости, отдельные служебные узлы, например чекер. Прикладной узел получает `SimulationContext` и обменивается сообщениями через абстракции симулятора, не обращаясь к очереди напрямую.
3. **Алгоритмы консенсуса и состояние KV-store** Реализации Raft и гибридного протокола живут в узлах сети: выбор лидера, репликация журнала и применение команд к локальному KV-store выражены через сообщения и таймеры слоя 2. Так протокол тестируется в той же модели отказов, что и сеть.
4. **Сборщик метрик** Обязка вокруг прогонов симуляции: сценарии, в т.ч. хаос-тесты - множественные запуски с контролируемой случайностью, агрегация показателей и сравнение базового Raft с гибридом на одних и тех же условиях.

Логика изложения работы Ниже курсовая в основном следует этому подъему уровней, но могут быть маленькие расхождения в угоду удобства повествования. Сначала детально рассматривается слой 1 (MDS), затем слой 2 - симулятор сети и сценариев. Содержательная часть слоя 3: распределенное KV-хранилище и основы Raft изложены отдельными разделами, затем следует отдельный раздел о гибридном варианте протокола на базе Raft. Все они относятся к одному прикладному уровню. Слой 4 описан в разделе о сборщике метрик. Сравнение прогонов, таблицы и выводы по экспериментам приведены далее по тексту.

7 Message-Driven System

Message-Driven System - это система, состоящая из менеджера сообщений, а также из набора взаимодействующих *узлов-акторов*, чье поведение это реакция на входящие *сообщения*: обработка сообщения может изменить локальное состояние узла и породить новые сообщения другим узлам.

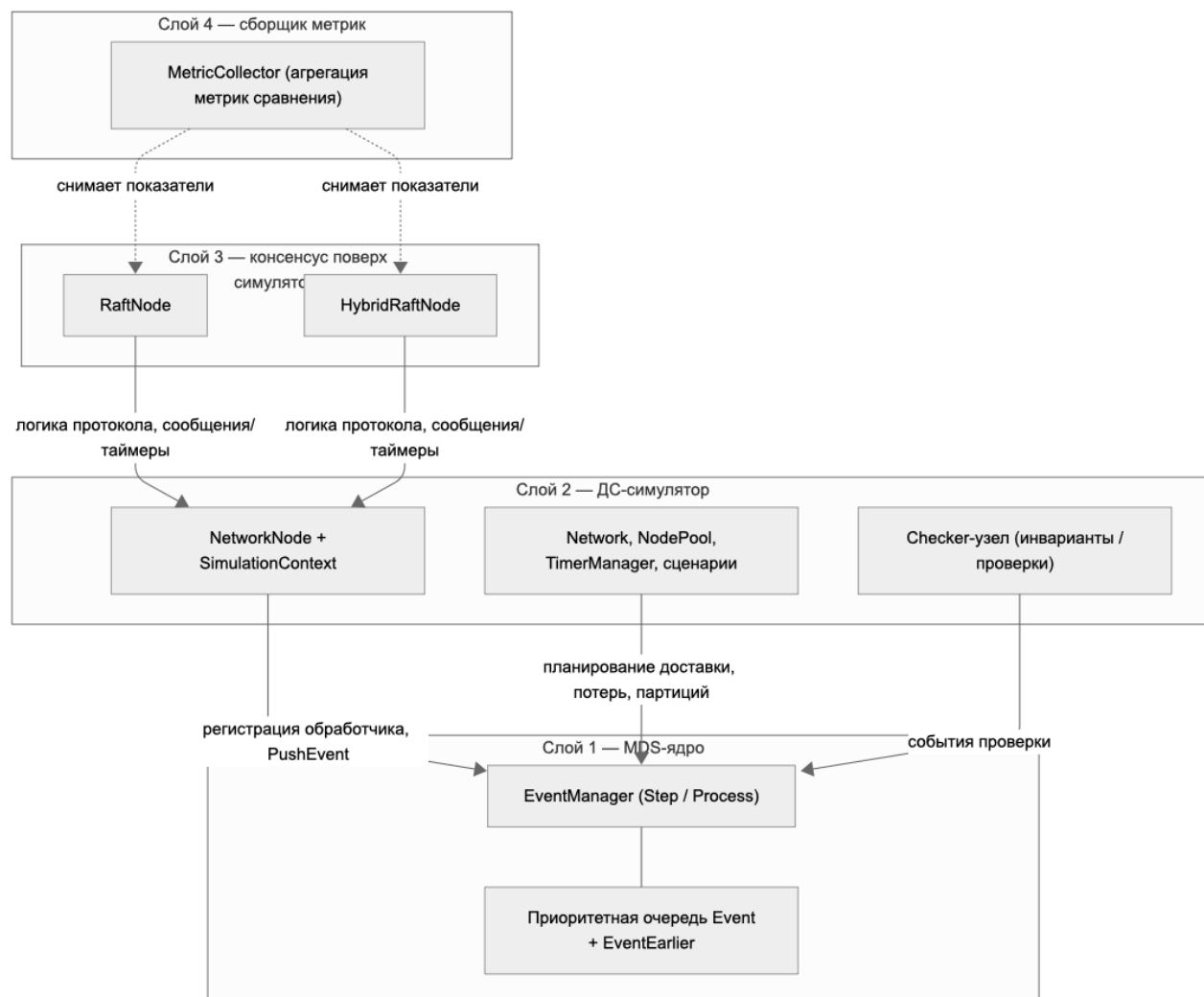


Рисунок 1: Слоистая архитектура среды: MDS, симулятор, KV-store и консенсус (Raft и гибрид), сбор метрик

7.1 Варианты MDS

Существует несколько вариантов архитектуры MDS:

- **Очередь на узел (mailbox)** У каждого актора своя FIFO-очередь входящих сообщений. Планировщик выбирает, с какого актора взять следующее сообщение.
- **Глобальная очередь событий** Все отложенные действия попадают в общий пул. Диспетчер извлекает элементы по правилу приоритета. У события явно задается *симуляционное время* как величина, сравнимая с другими моментами на той же шкале, и очередь упорядочивается по ней.
- **Вид доставки** Существует асинхронная и синхронная. В асинхронной модели отправка лишь ставит событие в очередь. В синхронной вызов обработчика может быть вложенным. Для воспроизводимости тестов чаще выбирают однопоточный цикл: извлечь событие → обработать → поставить в очередь новые события.

7.2 MDS в данной работе

Ниже зафиксирована *спецификация* MDS-ядра проекта (модуль `src/core`): что считается событием, как оно живет в очереди, как запускается обработка и какие гарантии дает менеджер. Общая идеология message-driven построения была подчерпана в [5].

Ядро и равноправные агенты На уровне MDS все узлы одинаковы по интерфейсу: каждый зарегистрированный узел - это пара, состоящая из идентификатора `NodeID` и функции-обработчика `EventHandler`. `NodeID` для каждого узла уникален - это идентификатор, по которому узлы могут адресовать друг друга. Его генерацией в рамках одной симуляции занимается `NodeIDManager`.

Диспетчер не различает разные виды узлов с разным поведением - различие появляется только в коде обработчика и в полезной нагрузке события, которую этот обработчик интерпретирует. У каждого зарегистрированного `NodeID` ровно один обработчик (`std::function`).

7.3 Формат события, контракт полей

Минимальный контракт записи в глобальной очереди таков:

1. `from` - идентификатор отправителя (`NodeID`).
2. `to` - идентификатор получателя. Диспетчер по нему ищет обработчик.
3. `timestamp` - *симуляционное* время, позволяющее упорядочить разные события между собой.
4. полезная нагрузка `data` - универсальный байтовый буфер.
5. `dispatch_order` - небольшое по месту в памяти целое число для детерминированного разрешения одновременных событий с равным `timestamp`.

Таким образом, письмо в MDS - это всегда структура с адресами, временем срабатывания и данными. Следующие уровни строятся на возможности сериализации и десериализации полезной нагрузки.

7.4 Время и порядок обработки

Вводится одна логическая шкала симуляционного времени. Глобальное текущее время `Now()` обновляется при извлечении события из очереди и принимает значение `timestamp` этого события. Очередь приоритетная - первым обрабатывается событие с наименьшим временем. При равенстве же времени порядок фиксируется сравнением `to`, затем `dispatch_order`, затем `from`, что обеспечивает *воспроизводимость* при тех же входных данных. Очередь реализована с нуля, в ее основе бинарная куча ввиду отсутствия нужного функционала у приоритетной очереди из стандартной библиотеки.

Постановка в очередь и исполнение (PushEvent, Step, Process)

- **PushEvent(Event e)** - только добавляет событие *e* в глобальную очередь, не вызывая обработчиков. Это операция планирования в момент **timestamp** события *e*.
- **Step()** - извлекает из очереди одно событие с наивысшим приоритетом, в частном случае с минимальным временем. Если очередь пуста, возвращается статус неполной операции. Иначе обновляются симуляционные часы, вызывается **HandleEvent** - для **event.to** ищется зарегистрированный обработчик и исполняется **handler(event, ctx)**, где **ctx** дает узлу доступ к **Now()**, собственному **NodeID** а также доступ к новому созданию событий через **ctx.PushEvent**. Также в аргументы **Step()** включен фактор завершения, принимающий в себя **Event** и возвращающий **bool** - нужно ли останавливаться. По умолчанию этот фактор возвращает всегда **false**.
- **Process()** - последовательно вызывает **Step()** до опустошения очереди (блокирующе в смысле однопоточного цикла: вызывающий код не вернется, пока очередь не станет пустой). В **Step()** фактор остается по умолчанию.
- **ProcessUntil(t)** - то же, но останавливается, если следующее событие было бы позже времени *t* (удобно для пошаговых сценариев и отладки). Это условие и содержит в себе фактор завершения.

Порождение новых событий Важно заметить - рекурсивного исполнения другого события внутри того же вызова обработчика нет - только постановка в очередь.

7.5 Контекст выполнения

Под *контекстом* здесь понимается объект, который диспетчер передает обработчику узла вместе с текущим событием и который задает допустимые действия на этом шаге, не открывая всю внутренность симулятора. Задача контекста - связать три вещи:

1. кто сейчас исполняется - свой идентификатор узла.
2. какое симуляционное время считается за сейчас.
3. как породить следующие события легально, не храня напрямую внутри узла **Observer** на **EventManager**, а также избегая спуфинга (подмены поля **from** другим **NodeID**). Мы не должны напрямую получать доступ к последнему - нужна прослойка между узлом и менеджером, что и решает контекст.

На уровне ядра это **CoreContext** - он хранит ссылку на **EventManager** и **NodeID** получателя обрабатываемого события. Через него доступны **Now()**, **GetOwnID()** и **PushEvent** для постановки новых записей в глобальную очередь. Таким образом узел не может подменить свой адрес: идентификатор берется из контекста, а не задается произвольно при отправке.

7.6 Регистрация узлов и адресация

Новый узел появляется в системе через **EventManager::AddNode(...)** и только через этот механизм: генерируется уникальный **NodeID** (с учетом поколения индекса при повторном использовании слотов), обработчик сохраняется в таблице **node_pool_**. Событие с **to**, для которого нет записи в таблице, молча игнорируется при доставке - это осознанное упрощение ядра: существование адресата обеспечивается инвариантами верхнего уровня (не посылать на уничтоженный **id**, не переиспользовать **id** без согласования). Отправитель **from** ядром не проверяется: корректность подстановки **from** - ответственность кода узла.

CoreContext при обработке сообщает узлу только его собственный идентификатор (получатель текущего события), что исключает ошибку обработать чужое событие на стороне API контекста.

7.7 Зачем такое ядро

Однопоточный цикл без разделяемого состояния между потоками упрощает отладку и гарантирует, что полный порядок шагов определяется очередью и генератором случайных чисел с фиксированным **seed** на верхних уровнях.

8 Симулятор

Когда описана MDS, использующаяся в данном проекте, самое время сделать надстройку над ней.

8.1 Новые компоненты

Основная идея - симуляция работы сети как сущности, способной терять, задерживать или дублировать сообщения. Также на этом слое появляются примитивы:

1. Создание и удаление *таймеров* для каждой ноды.
2. *Локальные сообщения*, способность отсылать и принимать их. Их философия в следующем - хочется иметь над каждым узлом пользователя этого узла, который может отсылать на узел какие-то команды, а узел в свою очередь отправляет результат обработчика этой команды. Причем на связь узла и пользователя не накладываются ограничения сети - ответ доходит мгновенно в обоих направлениях.

Возможность отправки и приема сообщений между узлами остается.

Также на этом слое появляются узлы со специальной ролью. **Network** принимает события об отправке, моделирует задержку и потерю и порождает событие доставки получателю. Без шлюза к **Network** межузловые сообщения не проходят через единую модель среды. Отдельно задается приемник локальных сообщений: при регистрации узла с тегом `NodeTag::kCHECKER` его идентификатор записывается в `checker_local_sink`, и безадресный вызов `SendLocal(msg)` доставляет сообщение именно туда - конкретный класс узла может быть любым, важен только тег при добавлении в сценарий. Благодаря чекеру легко можно проверять инварианты системы. Но логика чекера все равно находится в философии, описанной выше в локальном сообщении.

8.2 Модернизация контекста `SimulationContext`

Поверх `CoreContext` вводится `SimulationContext`: он переводит прикладные действия в события MDS. Основные операции такие:

1. отправка `Message` другому узлу через настроенный шлюз к узлу **Network** (межузловая доставка с задержками и потерями по правилам сети).
2. `SendLocal` - локальное сообщение узлу (без очереди в **Network**, для сценариев и пользователя).
3. постановка, отмена и проверка таймеров с учетом `SimulationTimerBook`, чтобы устаревшее срабатывание таймера не обработал узел после отмены.

Код протоколов (Raft, гибрид) опирается преимущественно на `SimulationContext`: алгоритм остается на уровне, где доступны общие процедуры вроде отправить сообщение или поставить таймер, а детали сети и сериализации полезной нагрузки сосредоточены в симуляторе. Для реализации Raft и его гибрида такого интерфейса системы достаточно.

8.3 Доставка через узел **Network**

Вызов `SendMessage` ставит в очередь событие-заявку на отправку. Если в `SimulationContextOptions` задан `network_gateway`, заявка маршрутизируется на узел **Network** как отдельное событие с адресатом-шлюзом. Обработчик **Network** по заявке со статусом `Sended` решает что делать с сообщением: с учетом глобальных параметров `NetworkSettings` (вероятность потери, диапазон задержки) и состояния конкретных узлов, описанного в `NodeNetworkSettings` (отказ, разбиение пары узлов). При отсутствии потери в очередь попадает событие с тем же `Message`, статусом доставки `Received` и временем прибытия `Now() + delay`. При потере генерируется событие о неуспехе, и оно никаким образом не обрабатывается.

Для одной пары $(from, to) \in \text{NodeID}^2$, где `NodeID` - множество всех выданных идентификаторов, время прибытия последовательных успешных доставок не убывает, чтобы порядок на канале не нарушался при равных или близких задержках.

На рисунке 2 показана схема взаимодействия отправителя, узла **Network** и получателя при отправке сообщения.

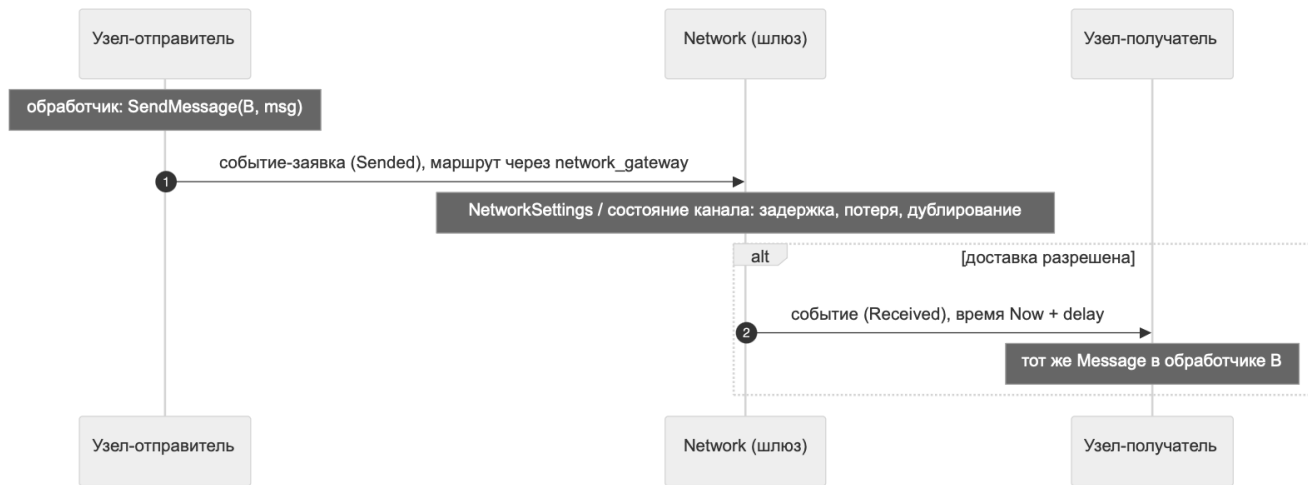


Рисунок 2: Диаграмма последовательности межузловой доставки

8.4 Две перегрузки SendLocal

Имеет смысл явно различать API. Вызов `SendLocal(msg)` без получателя отправляет локальное событие на узел из `checker_local_sink` (если он настроен и не совпадает с текущим узлом) - это путь от исполнителя к тестовому приемнику без участия `Network`. Вызов `SendLocal(to, msg)` в реализации совпадает с маршрутом `SendMessage(to, msg)`: через `PushApplicationMessage` и при наличии шлюза - через `Network` тоже подчиняется задержкам и потерям.

Сценарий может также планировать локальные сообщения извне через `ScheduleLocalMessage(at, to, logical_from,` порождая событие нужного типа в заданный момент времени (удобно для имитации клиента).

8.5 Сборка сценариев

Класс `SimulationScenario` объединяет `EventManager`, общие для всех узлов `SimulationContextOptions` и отдельную `SimulationTimerBook` на каждый `NodeID`.

Конструктор с конфигурацией `Network::Config` сразу регистрирует узел `Network` и прописывает его как `network_gateway`.

Метод `AddNode` сохраняет объект узла и регистрирует обертку: при каждом событии создается `SimulationContext` с ссылкой на книгу таймеров данного узла и вызывается пользовательский `OnSimulationEvent`.

Для юнит-тестов сценариев и хаос-тестов предусмотрены планировщики `ScheduleMessage`, `ScheduleNodeFail`, `ScheduleNodeRecover`, `SchedulePartitionPair`, а продвижение времени - `RunUntil`, оборачивающий `ProcessUntil` менеджера событий.

Ниже приведен минимальный фрагмент, иллюстрирующий порядок действий: конфигурация сети, создание `SimulationScenario`, регистрация узлов (в т.ч. с тегом `kCHECKER`), планирование первого сообщения и прогон до заданного времени. Классы узлов `EchoNode` и `InvariantChecker` здесь не показаны - они задают реакцию на `OnSimulationEvent`, но из названия ясно, что они отправляют обратно пришедшее им сообщение.

```

1
2 NetworkSettings net{.drop_prob = 0.f, .min_delay = 0.f, .max_delay = 0.f};
3 SimulationScenario sim(
4     Network::Config{.behavior = net, .random_seed = RandomSeed::kDEFAULT});
5
6 sim.AddNode(InvariantChecker{}, NodeTag::kCHECKER);
7 const NodeID a = sim.AddNode(EchoNode{});
8 const NodeID b = sim.AddNode(EchoNode{});
9
10 sim.ScheduleMessage(0.f, a, b, Message::FromDescription("hello", {}));
11 sim.RunUntil(10.f);

```

8.6 Вывод

Таким образом, над MDS-ядром собрана полноценная дискретно-событийная модель кластера. В распоряжении протоколов оказывается единый API **SimulationContext**: межузловой обмен через узел **Network** с настраиваемыми задержками, потерями, отказами узлов и *партициями* (разделениями сети на несвязные компоненты). Также в распоряжении и таймеры с отменой и проверкой актуальности, и локальные сообщения.

Сборка прогона сводится к **SimulationScenario**: регистрация узлов, планирование внешних воздействий и пошаговое или ограниченное по времени исполнение поверх того же **EventManager**, что сохраняет детерминированный порядок событий, заданный в разделе о ядре.

Далее этот контракт используется при описании и реализации Raft и гибридного протокола. Этого контракта достаточно для реализации алгоритмов консенсуса.

9 Распределенное KV-хранилище

В терминах репликации конечного автомата (см. введение) *KV-хранилище* в этой работе — частный случай автомата: *состояние* — отображение ключ $\rightarrow$ значение, а *команды* задают детерминированные переходы. Отдельный сетевой движок БД не выделяется: на каждом узле кластера есть свой объект KV Store.

Согласованность между узлами достигается тем, что все исправные реплики применяют одну и ту же последовательность мутаций в одном и том же порядке (за это ответственен консенсус).

Типы ключа и значения — параметры шаблона (**TKey**, **TVal**). Для обмена между узлами в симуляции они должны обязательно уметь сериализоваться в байтовый буфер.

9.1 Локальный контракт

Класс **KVStore** (модуль **kv/**) инкапсулирует одну реплику отображения в памяти, внутри - `std::map<TKey, TVal>`. Интерфейс:

- **Get(key)** — чтение: `std::optional<TVal>` с значением или пустой вариант, если ключа нет.
- **Set(key, value)** — запись или перезапись пары.
- **Delete(key)** — удаление значения. При отсутствии ключа возвращается ошибка, состояние не меняется.

Операция **Get** не изменяет состояние автомата. **Set** и **Delete** — *мутации*. Распределенность KV — не методы одного экземпляра, а согласованное повторение мутаций на нескольких экземплярах **KVStore**.

9.2 Инвариант согласованности

Если все реплики стартуют из одного начального состояния и получают одну и ту же последовательность мутаций **eSET/eDEL** в одном порядке, то после каждого шага их **KVStore** совпадают. Задача консенсуса - гарантировать именно такую общую последовательность при отказах и задержках. Следующий раздел вводит для этого журнал, роли узлов и правила коммита.

10 Основы консенсуса Raft

10.1 Контекст в симуляторе

В прогоне участвуют n узлов с логикой Raft, каждый зарегистрирован в MDS как обычный узел, плюс отдельный узел **Network**. В тестах можно увидеть разные варианты узлов чекера - они проверяют разные инварианты Raft.

Клиент в этой модели это отдельный узел кластера - чекер, но он выступает как внешний источник запросов к узлу Raft (в тестах и сценариях - локальное сообщение). Узел получает команду для KV, решает, лидер он или нет, и либо обрабатывает, либо пересылает текущему лидеру по тем же **Message**.

10.2 Типы состояния узлов в Raft

У каждого узла в любой момент ровно одна *роль* - это часть локального состояния. Роли переключаются при разборе входящих сообщений или при срабатывании таймеров.

Три роли:

1. **Follower** - пассивная реплика: принимает от лидера репликацию журнала, отвечает на запросы голосования, сама не инициирует запись в журнал. Типичный переход: по истечению election timeout без признаков живого лидера - узел переходит в состояние Candidate.
2. **Candidate** - участник выборов: временно увеличивает терм, рассылает сообщение с тегом **RequestVote**, собирает большинство или сдается по таймауту или сообщению от лидера. Переход: большинство голосов - в Leader. Валидное сообщение от лидера с подходящим термом - обратно в Follower.
3. **Leader** - в данном терме единственный узел (важно), который добавляет новые записи в журнал от клиентских мутаций и тянет за собой фолловеров через сообщение с тегом **AppendEntries** и heartbeat-ы. Переход в Follower - например, при получении сообщения с большим термом или когда протокол требует признать другого лидера.

Детальный разбор входов (какие именно сообщения и таймеры вызывают какие реакции) дан ниже в подразделе о локальном поведении узла.

10.3 Что решает протокол и какие у него гарантии

Raft [3] - протокол распределенного консенсуса: правила обмена сообщениями и обновления локального состояния на каждом узле, при соблюдении которых все исправные реплики приходят к одному и тому же упорядоченному журналу записей (в данной работе в записях лежат **Command** для **KVStore**, см. раздел о KV).

Важно ввести еще несколько терминов: *Журнал* на узле - упорядоченная последовательность записей, *индекс записи* - ее позиция в этой последовательности.

Пусть f - допустимое число узлов, вышедших из строя по модели отказа с остановкой - узел перестает отвечать, не шлет произвольные ложные сообщения. При $n > 2f$ протокол обеспечивает:

1. безопасность - если две реплики применили запись с одним и тем же индексом журнала, то их содержимое совпадает.
2. живучесть при связном большинстве и стабильном лидере - новые команды в конечном счете попадают в журнал и коммитятся.

Терм (term): монотонный номер *эпохи* (раунде между перевыборами лидера, нужна чтобы старые сообщения не влияли на текущий процесс). В каждом терме избранный лидер единственный, смена лидера сопровождается ростом терма.

10.4 Сообщения протокола и команды KV

Введем термин *heartbeat* - периодические сообщения от лидера к фолловерам (часто в виде **AppendEntries** без новых команд), подтверждающие, что лидер жив. Если фолловер не получает ни heartbeat, ни других сообщений лидера дольше *election timeout*, он считает лидера недоступным и начинает выборы.

Между узлами шлются сообщения фиксированных видов - голосование **RequestVote**, репликация **AppendEntries** и команды на KV **client_command**. *Команда* в смысле раздела про KV - содержимое записи журнала. Сообщение с тегом **AppendEntries** может нести одну или несколько записей журнала, не каждое сетевое сообщение добавляет новую команду в журнал (например, пустой heartbeat).

10.5 Функциональная роль узлов

Follower Принимает от лидера **AppendEntries**, обновляет журнал по правилам согласования хвоста - поля вроде **prevLogIndex** или **prevLogTerm** в сообщении проверяют согласованность с локальным журналом. Отвечает на **RequestVote**. По таймеру election timeout переходит в Candidate.

Candidate Увеличивает терм, голосует за себя, рассылает **RequestVote**. При голосах строгого большинства из n становится Leader. При отказе (таймаут) может начать новый раунд выборов. Если приходит сообщение с термом не ниже текущего от легитимного лидера - откатывается в Follower.

Leader Принимает ответы на `AppendEntries`, обновляет, какие индексы журнала известны большинству - *commit index*. Шлет фолловерам heartbeat и догоняющие `AppendEntries`. Обработывает клиентские мутации: добавляет запись в свой журнал, затем добывается ее копирования на большинство узлов. Индекс становится *закоммиченным*, после чего запись допускается к применению к `KVStore`. Чтение `Get` в реализации проекта обслуживается на лидере без новой записи в журнале.

10.6 Выборы и условие голоса

Кандидат шлет `RequestVote` с текущим термом и кратким описанием своего журнала. Узел отдает голос только если одновременно выполнены все условия:

- терм кандидата не меньше собственного термина узла.
- в этом терме при выборах узел еще не отдал голос другому кандидату.
- журнал кандидата не старше журнала узла в смысле Raft: сравнивают терм последней записи и ее индекс. Сначала смотрят на терм, при равных терминах сравнивают длину журнала. Кандидат получает голос только если его последняя запись не старше последней записи узла по этому правилу.

10.7 Репликация и коммит

Репликация записи означает: лидер уже вставил запись в свой журнал и добился того, что та же запись появилась в журналах у более половины узлов благодаря успешным `AppendEntries` и ответам фолловеров. Тогда индекс помечается закоммиченным, а именно узлы сдвигают `commitIndex` и применяют новые записи к `KVStore` по порядку и обновляют `lastApplied` - индекс последней примененной операции.

10.8 Данные в узле и таймеры

У каждого узла хранятся, в частности: номер текущего термина, кому отдан голос в этом терме, журнал, индекс последней закоммиченной и последней примененной к хранилищу записи. У лидера дополнительно, на каждого фолловера, хранятся числа `nextIndex` - с какого индекса журнала слать дальше - и `matchIndex` - до какого индекса фолловер подтвердил совпадение.

Таймеры выставляются через `SimulationContext`: случайный `election timeout` в заданном диапазоне на узле и период heartbeat у лидера. Персистентность журнала на диск и перезапуск процесса в объеме симуляции не моделируются: состояние узла живет в памяти на протяжении прогона, но этого достаточно.

11 Гибридный протокол на базе Raft

В проекте реализован узел `HybridRaftNode` с гибридным вариантом протокола Raft. Выборы лидера, работа с клиентом, запрос `AppendEntries` и инварианты журнала совпадают с описанным в предыдущем разделе. Меняется только политика согласования хвоста журнала после отказа фолловера, когда не сходятся `prevLogIndex` и `prevLogTerm`.

В базовой реализации лидер при ответе с ошибкой уменьшает `nextIndex` для этого фолловера на единицу и снова шлёт `AppendEntries`. При большом расхождении журналов это даёт много шагов.

В гибридном варианте ответ фолловера расширен полями `conflict_term` и `conflict_index`: по ним лидер сразу сдвигает `nextIndex` к началу конфликтующего термина или к указанному индексу. Тип сообщения ответа в симуляции - `HybridAppendEntriesResponse`, формат сериализации вынесен в `hybrid_message_specs.h`. При отказе лидер дополнительно шлёт локальное служебное сообщение для учёта в метриках; это используется слоем сборщика метрик при сравнении прогонов, об этом будет ниже.

Модель узла гибрида по-прежнему остановка, а цель изменений - меньше сообщений при синхронизации логов при тех же гарантиях безопасности, а также снижение среднего времени ответа на запросы клиентов.

12 Сборщик метрик

Слой 4 не меняет прежние правила Raft и не вмешивается в **Network**: он только наблюдает прогон симуляции и накапливает статистику для сравнения базового Raft и гибрида на одних и тех же сценариях. Реализация лежит в директории проекта `tests/metrics/` и связана с хаос-сценариями в `tests/mc/`.

12.1 Интерпретация в симуляторе

Как и для других проверок, сборщик регистрируется в `SimulationScenario` с тегом `kCHECKER`: у него тот же интерфейс `OnSimulationEvent`, что у узла `Raft`. На него попадают локальные сообщения. Среди них - ответы клиенту `client_command_response` с сериализованным `command::ResponsePayload` и служебное локальное сообщение `raft_metric_append_entries_reject`, которое лидер шлёт при отклонении `AppendEntries` фолловером. Такое происходит и в базовом `Raft`, и в гибриде. Таким образом, метрики снимаются с потока событий `MDS`, а не опросом памяти чужих узлов.

12.2 План клиентских операций и задержки

Для измерения задержек нужна временная метка запроса. В прогоне задаётся общий для теста вектор `ScheduledClientOp`: пара, состоящая из симуляционного времени `at` и команды `Command`. Сборщик при получении успешного `client_command_response` сопоставляет ответ с первой ещё не сопоставленной записью с тем же типом, ключом и (для `SET`) значением, разница таймстампа события ответа и `op.at` попадает в выборку задержек.

Структура `RaftMcMetricsReport` хранит счётчик отказов репликации `append_entries_rejects` и массивы задержек `set_latencies`, `get_latencies`, вспомогательные методы усредняют выборки. Коллектор заполняет журнал всех ответов `response_log` для отладки линейаризуемости.

12.3 Сборка сценария

Шаблон `BuildRaftMcSimulation` создаёт `SimulationScenario` с заданным `Network::Config`, регистрирует экземпляр `RaftMcMetricsCollector` как чекер и добавляет `num_nodes` узлов с шаблонным типом узла `Raft` (`RaftNode` или `HybridRaftNode`). Идентификаторы узлов консенсуса возвращаются наружу. Далее тест подключает к ним сеть `WireRaftCluster`, планирует хаос сценарий и клиентские операции и вызывает `RunUntil`. После прогона достаточно снять `SnapshotReport` с коллектора. Он и будет хранить запакованные метрики прогона.

12.4 Связь с хаос-тестами

Модуль `tests/mc/` задаёт расписания - партии, задержки - при фиксированном `seed` и запускает пару прогонов - `vanilla` и гибрида - с общим телом сценария. Сборщик метрик даёт сопоставимые скаляры и распределения задержек, обработка перцентилей и проверка инвариантов после хаоса остаются в коде сценария. Итоговые таблицы и интерпретация чисел выносятся в раздел результатов экспериментов, об его интерпретации речь пойдет ниже.

13 Эксперименты

Используется 5 рафт-узлов при каждом прогоне.

Хаос-сценарий из директории `tests/mc/` - серия `SET/GET`, расписание помех сети - прогоняется дважды для одного и того же `seed` генератора: с узлами `RaftNode` и `HybridRaftNode`. Ниже на рисунке 3 - фрагмент вывода теста: два режима сети в одном запуске - только джиттер задержек без потерь и джиттер с вероятностью потери пакетов. Задержки в единицах симуляционного времени (`TTime`), среднее и 95-й перцентиль по выборкам успешных ответов.

В обоих режимах у гибрида существенно меньше отказов `AppendEntries` по метрике `append_entries_rejects`. Средняя задержка `SET` ниже у гибрида. При потерях разница по `SET` и `GET` наиболее заметна. Для `GET` в режиме только задержек среднее у гибрида выше - на интерпретацию влияет форма распределения и сопоставимость фаз сценария, это может быть видно из несимметричного перцентиля 95.

Также количество ОК ответов у гибрида выше, чем у `vanilla`. Это еще один знак того, что гибрид эффективнее `vanilla` по метрикам.

13.1 Множественный запуск и сборка статистики

Чтобы оценить устойчивость выводов к случайной дисперсии, то же расписание `SET/GET` и помехи при фиксированных параметрах модели сети выполнялось сто раз с разными сидами. Для каждого прогона из метрик извлекались средние задержки по успешным `SET` и `GET`. Результаты на рисунке 4.

```

artyomkononov@Artyoms-MacBook-Pro build % ./test_mc
Randomness seeded to: 1731980800
--- Monkey chaos: jitter only (no drops) ---

--- Monkey chaos metrics: Raft vs HybridRaft ---
metric Raft HybridRaft
-----
append_entries_rejects 302 50
SET samples (ok latencies) 298 299
GET samples (ok latencies) 300 300
mean SET latency 87.6188 64.1060
p95 SET latency 324.8191 250.1777
mean GET latency 28.3908 30.6393
p95 GET latency 55.4137 149.5195
-----

--- Monkey chaos: jitter + packet loss ---

--- Monkey chaos metrics: Raft vs HybridRaft ---
metric Raft HybridRaft
-----
append_entries_rejects 287 50
SET samples (ok latencies) 297 299
GET samples (ok latencies) 295 300
mean SET latency 317.7247 146.0748
p95 SET latency 424.0469 287.7289
mean GET latency 317.0872 93.9901
p95 GET latency 834.6211 435.5055
-----

All tests passed (17 assertions in 3 test cases)

```

Рисунок 3: Иллюстрация к экспериментам: длинный хаос-сценарий (в т.ч. 300 SET и 300 GET). Сравнение условий без потерь/с потерями. Оба условия с задержкой пакетов.

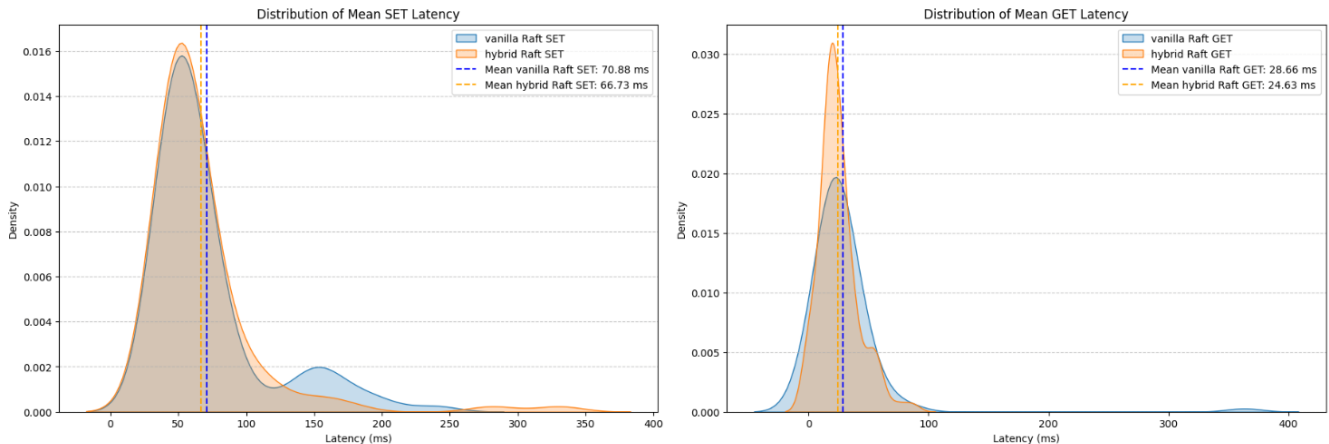


Рисунок 4: Результат эксперимента со сбором метрик со 100 запусков с разными ключами и составленный график распределения времен задержки. Оценка плотности (KDE) распределений средней задержки по выборке успешных ответов внутри каждого прогона. Синий - базовый Raft, оранжевый - гибрид; пунктир - среднее по 100 значениям средних задержек.

Слева - распределение средних задержек SET: по оси абсцисс - значение средней задержки SET в одном прогоне, по вертикали - сглаженная оценка плотности (KDE). Синяя кривая соответствует базовому Raft, оранжевая - гибриду; вертикальные пунктиры того же цвета отмечают среднее средних. Оранжевая кривая и ее пунктир сдвинуты влево. Это означает, что в типичном прогоне гибрид чаще даёт меньшую среднюю задержку записи.

Справа - то же для средних задержек GET: KDE по ста прогонам и пунктирные средние. Здесь наглядно видно сопоставление также и вариативности: если распределение гибрида сосредоточено левее и его среднее ниже, это указывает на более быстрые чтения в среднем по выборке прогонов.

В совокупности этим экспериментом мы дополнили табличный снимок одного запуска (рисунок 3). Показано, как ведут себя средние задержки при переборе случайности. Таким образом построена оценка Монте Карло.

14 Заключение

В работе получена среда для воспроизводимых экспериментов с консенсусом: реализованы MDS-ядро и дискретно-событийный симулятор с моделью сети, KV-слой и два варианта Raft - базовый и гибридный, тесты и сборщик метрик, хаос-сценарии для сравнения прогонов на одинаковых входах.

С архитектурной точки зрения четыре слоя из раздела об общей структуре стыкуются в одном репозитории: события и порядок шагов задаётся MDS, симулятор добавляет `Network`, таймеры и сценарии, узлы Raft и гибрид исполняют протокол поверх `SimulationContext`, сборщик метрик на узле-приёмнике снимает метрики без изменения логики протокола.

Количественные выводы из секции "Эксперименты" можно разделить на два уровня. По одиночному прогону (рисунок 3) гибрид даёт существенно меньше отказов репликации (`append_entries_rejects`). При появлении потерь пакетов разница по задержкам SET и GET наиболее заметна. Отдельный прогон не отменяет случайной дисперсии.

Агрегированная выборка из ста прогонов (рисунок 4) показывает распределения средних задержек. Для GET оценка плотности и среднее по выборке смещены в сторону меньших задержек у гибрида, то есть чтения в среднем быстрее и стабильнее выигрывают у гибридного варианта на этом наборе сценариев и наборов сидов. Для SET мы видим дисперсию между прогонами, и у гибрида она меньше. Вместе с меньшим числом отказов `AppendEntries` это согласуется с тем, что гибрид реже вынужден догонять журнал шагами в 1 по индексу лога.

Итог работы: в рамках реализованного симулятора и описанного хаос-сценария гибридная политика согласования хвоста журнала предпочтительнее базовой Raft по сводным метрикам задержек на выборке из многих ключей и по числу отказов `AppendEntries`, многократные прогоны подтверждают эту гипотезу.

Актуальная версия реализации, воспроизводимая сборка примеров, тестов и метрик, и сопутствующие материалы находятся в репозитории [7].

15 Дальнейшее развитие

Возможное продолжение работы - перенос узлов на реальные процессы и обмен по TCP, персистентность журнала на диск, другие сценарии хаос тестов.

Список литературы

- [1] Miguel Castro and Barbara Liskov. Practical byzantine fault tolerance. In *OSDI*, pages 173–186, 1999. URL: <https://pmg.csail.mit.edu/papers/osdi99.pdf>.
- [2] Leslie Lamport. Paxos made simple. *ACM SIGACT News*, 32(4):51–58, 2001. URL: <https://lamport.azurewebsites.net/pubs/paxos-simple.pdf>.
- [3] Diego Ongaro and John Ousterhout. In search of an understandable consensus algorithm. In *USENIX Annual Technical Conference*, pages 305–319, 2014. URL: <https://raft.github.io/raft.pdf>.
- [4] OpenAI. Chatgpt, 2026. Веб-интерфейс генеративной модели. URL: <https://chatgpt.com>.
- [5] Trushin D. V. C++ lecture notes (main.pdf). 2026. URL: <https://github.com/DimaTrushin/CppCode/blob/master/main.pdf>.

- [6] Maarten van Steen and Andrew S. Tanenbaum. *Distributed Systems*. Maarten van Steen, 3rd edition, 2017. URL: <https://www.distributed-systems.net/>.
- [7] Konovalov A. Y. Environment for distributed system. 2025. URL: <https://github.com/aikonovalov/DistSysEnv>.